Technical Note: Model Predictive Path Integral Control — Derivation and Implementation

Ruixiang Du
ruixiang.du@gmail.com

Abstract

This note derives the MPPI control update implemented in xmNavigation, following the information-theoretic formulation, and records the implementation decisions (warm start, baseline subtraction, sampling variants, control parameterization) with references to the originating literature. It closes with the CPU execution strategy and the planned CUDA extension for discrete GPU and Jetson Orin platforms.

1 MPPI

1.1 Lineage

Path-integral optimal control originates with Kappen [1], who showed that for control-affine stochastic systems with quadratic control cost the Hamilton–Jacobi–Bellman equation becomes linear under an exponential transformation of the value function, so the optimal control admits a Feynman–Kac path-integral representation evaluable by Monte Carlo. Theodorou et al. generalized the framework to parameterized policies as PI² [2]. Williams et al. brought it to receding-horizon control as MPPI, first with the path-integral derivation [3], [4], then re-derived it from an information-theoretic argument [5], [6] that removes the control-affine restriction — the dynamics only need to be **sampleable**. This note follows the information-theoretic derivation; the algorithm implemented in `src/control/mppi` is the algorithm of [6].

1.2 Problem setup

Consider discrete-time dynamics with state $\boldsymbol{x}_t \in \mathbb{R}^n$ and commanded control $\boldsymbol{v}_t \in \mathbb{R}^m$:

$$\boldsymbol{x}_{t+1} = \boldsymbol{F}(\boldsymbol{x}_t, \boldsymbol{v}_t) \tag{1}$$

where the **executed** control is the commanded control corrupted by (or deliberately injected with) Gaussian noise: $\boldsymbol{v}_t = \boldsymbol{u}_t + \boldsymbol{\varepsilon}_t$, $\boldsymbol{\varepsilon}_t \sim \mathcal{N}(\mathbf{0}, \boldsymbol{\Sigma})$. A trajectory $\tau = (\boldsymbol{x}_0, \boldsymbol{v}_0, ..., \boldsymbol{v}_{T-1})$ accrues the state cost

$$S(\tau) = \varphi(\boldsymbol{x}_T) + \sum_{t=0}^{T-1} q(\boldsymbol{x}_t) \tag{2}$$

with terminal cost φ and running cost q. Note S carries **no control cost**: the control effort penalty will emerge from the derivation as a KL term.

1.3 Free energy and the optimal distribution

Let $\mathbb{P}$ denote the **base** distribution over trajectories induced by running the noise alone ($\boldsymbol{u} = \mathbf{0}$), and $\mathbb{Q}_{\boldsymbol{u}}$ the distribution induced by a control sequence $\boldsymbol{u} = (\boldsymbol{u}_0, ..., \boldsymbol{u}_{T-1})$. Define the **free energy** of the control problem [6]:

$$\mathcal{F}(S) = -\lambda \log \mathbb{E}_{\mathbb{P}}\left[\exp\left(-\frac{1}{\lambda} S(\tau)\right)\right] \tag{3}$$

where $\lambda > 0$ is the **inverse temperature**. Jensen's inequality gives, for any distribution $\mathbb{Q}$ absolutely continuous with $\mathbb{P}$:

$$\mathcal{F}(S) \leq \mathbb{E}_{\mathbb{Q}}[S(\tau)] + \lambda \ \mathrm{KL}(\mathbb{Q} \parallel \mathbb{P}) \tag{4}$$

i.e. the free energy lower-bounds the stochastic optimal control objective (state cost plus a control penalty in the form of a KL divergence from the base distribution). The bound is tight — Equation 4 holds with equality — for the **optimal distribution** $\mathbb{Q}^*$ defined by the Gibbs/ Boltzmann density

$$\frac{d\mathbb{Q}^*}{d\mathbb{P}}(\tau) = \frac{\exp\left(-\frac{1}{\lambda} S(\tau)\right)}{\mathbb{E}_{\mathbb{P}}\left[\exp\left(-\frac{1}{\lambda} S(\tau)\right)\right]} \tag{5}$$

$\mathbb{Q}^*$ is not directly realizable by a controller. MPPI therefore picks the realizable distribution closest to it: minimize $\mathrm{KL}(\mathbb{Q}^* \parallel \mathbb{Q}_{\boldsymbol{u}})$ over the mean-shifted Gaussian family. For Gaussian noise this minimization has the closed-form solution (moment matching)

$$\boldsymbol{u}_t^* = \mathbb{E}_{\mathbb{Q}^*}[\boldsymbol{v}_t] \tag{6}$$

1.4 Importance sampling and the MPPI weights

Equation 6 is an expectation under $\mathbb{Q}^*$, which we can only evaluate by importance sampling from a distribution we **can** roll out — in practice $\mathbb{Q}_{\hat{\boldsymbol{u}}}$, the distribution induced by the previous solution $\hat{\boldsymbol{u}}$ (warm start). Writing the expectation under $\mathbb{Q}_{\hat{\boldsymbol{u}}}$ and collecting the Radon–Nikodym factors of the Gaussian mean shift yields the **importance-sampling-corrected** trajectory cost [4], [6]:

$$\tilde{S}(\tau_k) = S(\tau_k) + \gamma \sum_{t=0}^{T-1} \hat{\boldsymbol{u}}_t^T \boldsymbol{\Sigma}^{-1} \boldsymbol{\varepsilon}_{t,k} \tag{7}$$

where $\boldsymbol{\varepsilon}_{t,k}$ is the k-th sampled noise sequence and $\gamma = \lambda(1-\alpha)$ exposes the **control-cost decoupling parameter** $\alpha \in [0, 1]$ introduced in [6] ($\alpha = 0$: full KL penalty toward the base distribution; $\alpha = 1$: no pull toward zero control). With K rollouts, the weights and update are

$$\rho = \min_k \tilde{S}(\tau_k), \quad w_k = \frac{\exp\left(-\frac{1}{\lambda}\left(\tilde{S}(\tau_k) - \rho\right)\right)}{\sum_{j=1}^{K} \exp\left(-\frac{1}{\lambda}\left(\tilde{S}(\tau_j) - \rho\right)\right)} \tag{8}$$

$$\boldsymbol{u}_t \leftarrow \hat{\boldsymbol{u}}_t + \sum_{k=1}^{K} w_k \boldsymbol{\varepsilon}_{t,k} \tag{9}$$

The baseline subtraction ρ leaves Equation 8 mathematically unchanged and is mandatory numerically (it prevents uniform exponent underflow) [6].

1.5 The algorithm

1. Shift the previous solution one step (executed step removed, tail padded); this is the sampling mean $\hat{\boldsymbol{u}}$.

2. Sample K noise sequences $\boldsymbol{\varepsilon}_k$; form perturbed controls $\boldsymbol{v}_k = \hat{\boldsymbol{u}} + \boldsymbol{\varepsilon}_k$ and clamp to actuator bounds.
3. Roll out the dynamics for each $\boldsymbol{v}_k$; accumulate Equation 2 and the correction of Equation 7.
4. Compute weights Equation 8 with baseline subtraction; apply the update Equation 9.
5. Optionally smooth the resulting sequence (Savitzky–Golay [7], or use a smoothness-preserving sampler, § below).
6. Execute $\boldsymbol{u}_0$; repeat.

The loop is a single optimization iteration per control step; the warm start is what makes this sufficient in practice [4].

1.6 Practical considerations

- **Temperature** λ sharpens ($\lambda \to 0$, winner-take-all) or flattens (large λ) the weights; it divides cost **differences**, so it is re-tuned whenever cost scales change [6].
- **Noise covariance** $\boldsymbol{\Sigma}$ is the exploration knob and defines the KL anchor; it must be full-rank on sampled channels since $\boldsymbol{\Sigma}^{-1}$ appears in Equation 7.
- **Failure modes and their literature**: sample impoverishment after disturbances (Tube-MPPI [8], Robust-MPPI [9]); control chattering from i.i.d. noise (Smooth-MPPI input lifting [10], low-frequency/colored sampling [11]); constraint handling — penalties inside rollouts are soft, so safety-critical constraints require an output-stage shield (control-barrier-function filtering [12]).
- **Sampling variants** implemented behind the sampler seam: normal–log-normal mixtures for cluttered spaces [13], colored noise [11], and annealed covariance schedules in the style of DIAL-MPC [14].
- **Control parameterization**: sampling spline knots instead of per-step controls reduces the decision dimension by an order of magnitude and enforces smoothness by construction; it is the enabling technique in whole-body legged sampling MPC [15], [16] and is a first-class option here.

1.7 Implementation notes (CPU now, CUDA next)

The CPU implementation follows the strategy validated by the Nav2 MPPI controller [7]: batch-major contiguous storage (structure-of-arrays over the K rollouts) that Eigen auto-vectorizes, preallocated workspace (no allocation in the control path), compile-time state/control dimensions via templates, and deterministic seeded sampling. Published throughput for this approach is roughly 30–60 million sample-timesteps per second per modern x86 core, which covers wheeled platforms (1–3k samples, 50–100 Hz) on a single core.

The rollout phase — clamp, step, accumulate cost — is extracted behind a backend seam (`rollout_backend.hpp`): sample k reads shared inputs and writes only S_k, so backends may evaluate the K rollouts in any parallel arrangement as long as the per-sample operation order is preserved (bitwise-reproducible results for any worker count; enforced by test). The CPU backend runs serially by default or over a pre-allocated worker pool with contiguous chunking. Measured on an 8-core x86 host at $K = 2048$: the SRB quadruped plan drops from 11.0 ms serial to 5.9 ms with 4 workers and 5.2 ms with 8; cheap wheeled models gain little (9.0 to 7.2 ms) because sequential noise generation dominates their budget (Amdahl). Per-sample counter-seeded noise streams would lift that cap and are the natural CPU follow-up; on the GPU path noise is generated on-device.

The CUDA extension for discrete GPUs and Jetson Orin implements the same backend seam and follows the architecture of MPPI-Generic [17]: samples across CUDA blocks, dynamics/ cost as header-only functors over raw spans so **the same functor code** compiles for both backends (avoiding the dual-implementation burden MPPI-Generic documents), and split-vs-fused rollout kernels selected by benchmark. The GPU path becomes necessary beyond roughly 16k samples per step, for neural-network dynamics, or for full-order legged rollouts [14].

Bibliography

[1] H. J. Kappen, "Linear Theory for Control of Nonlinear Stochastic Systems," *Physical Review Letters*, vol. 95, p. 200201, 2005, doi: 10.1103/PhysRevLett.95.200201.

[2] E. Theodorou, J. Buchli, and S. Schaal, "A Generalized Path Integral Control Approach to Reinforcement Learning," *Journal of Machine Learning Research*, vol. 11, pp. 3137–3181, 2010.

[3] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, "Aggressive Driving with Model Predictive Path Integral Control," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2016, pp. 1433–1440. doi: 10.1109/ICRA.2016.7487277.

[4] G. Williams, A. Aldrich, and E. A. Theodorou, "Model Predictive Path Integral Control: From Theory to Parallel Computation," *Journal of Guidance, Control, and Dynamics*, vol. 40, no. 2, pp. 344–357, 2017, doi: 10.2514/1.G001921.

[5] G. Williams *et al.*, "Information Theoretic MPC for Model-Based Reinforcement Learning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2017, pp. 1714–1721. doi: 10.1109/ICRA.2017.7989202.

[6] G. Williams, P. Drews, B. Goldfain, J. M. Rehg, and E. A. Theodorou, "Information-Theoretic Model Predictive Control: Theory and Applications to Autonomous Driving," *IEEE Transactions on Robotics*, vol. 34, no. 6, pp. 1603–1622, 2018, doi: 10.1109/TRO.2018.2865891.

[7] S. Macenski, T. Moore, D. V. Lu, A. Merzlyakov, and M. Ferguson, "From the Desks of ROS Maintainers: A Survey of Modern and Capable Mobile Robotics Algorithms in the Robot Operating System 2." 2023.

[8] G. Williams, B. Goldfain, P. Drews, K. Saigol, J. M. Rehg, and E. A. Theodorou, "Robust Sampling Based Model Predictive Control with Sparse Objective Information," in *Robotics: Science and Systems XIV*, 2018. doi: 10.15607/RSS.2018.XIV.042.

[9] M. Gandhi, B. Vlahov, J. Gibson, G. Williams, and E. A. Theodorou, "Robust Model Predictive Path Integral Control: Analysis and Performance Guarantees," *IEEE Robotics and Automation Letters*, vol. 6, no. 2, pp. 1423–1430, 2021.

[10] T. Kim, G. Park, K. Kwak, J. Bae, and W. Lee, "Smooth Model Predictive Path Integral Control Without Smoothing," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 10406–10413, 2022.

[11] B. Vlahov, J. Gibson, D. D. Fan, P. Spieler, A.-a. Agha-mohammadi, and E. A. Theodorou, "Low Frequency Sampling in Model Predictive Path Integral Control," *IEEE Robotics and Automation Letters*, vol. 9, no. 5, 2024, doi: 10.1109/LRA.2024.3382530.

[12] J. Yin, C. Dawson, C. Fan, and P. Tsiotras, "Shield Model Predictive Path Integral: A Computationally Efficient Robust MPC Method Using Control Barrier Functions," *IEEE Robotics and Automation Letters*, vol. 8, no. 11, 2023.

[13] I. S. Mohamed, K. Yin, and L. Liu, "Autonomous Navigation of AGVs in Unknown Cluttered Environments: log-MPPI Control Strategy," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 10240–10247, 2022.

[14] H. Xue, C. Pan, Z. Yi, G. Qu, and G. Shi, "Full-Order Sampling-Based MPC for Torque-Level Locomotion Control via Diffusion-Style Annealing." 2024.

[15] T. Howell, N. Gileadi, S. Tunyasuvunakool, K. Zakka, T. Erez, and Y. Tassa, "Predictive Sampling: Real-time Behaviour Synthesis with MuJoCo." 2022.

[16] J. Alvarez-Padilla, J. Z. Zhang, S. Kwok, J. M. Dolan, and Z. Manchester, "Real-Time Whole-Body Control of Legged Robots with Model-Predictive Path Integral Control." 2024.

[17] B. Vlahov, J. Gibson, M. Gandhi, and E. A. Theodorou, "MPPI-Generic: A CUDA Library for Stochastic Trajectory Optimization." 2024.